

POO em TypeScript — Aprenda Fazendo

Um resumo rápido + 10 exercícios progressivos para você resolver sozinho.

Como usar este PDF

A ideia é simples: leia o resumo e o exemplo inicial, depois tente resolver os 10 exercícios sem olhar o gabarito. As dicas existem para destravar, não para entregar a solução. O gabarito fica no final para você comparar sua implementação.

1. POO em poucas palavras

Programação Orientada a Objetos organiza o código em objetos que possuem dados (atributos) e comportamentos (métodos). Em TypeScript, classes são uma das principais formas de modelar esses objetos.

Classe: um molde para criar objetos.

Objeto: uma instância de uma classe.

Propriedade: um dado pertencente ao objeto.

Método: uma ação que o objeto sabe executar.

Construtor: usado para inicializar o objeto.

Encapsulamento: controla o acesso ao estado interno usando public/private/protected.

Herança: uma classe pode reaproveitar características de outra usando extends.

Polimorfismo: objetos de classes diferentes podem responder ao mesmo método de maneiras diferentes.

2. Exemplo simples

```
class Pessoa {
  constructor(
    public nome: string,
    private idade: number
  ) {}

  apresentar(): void {
    console.log(`Olá, eu sou ${this.nome}.`);
  }

  fazerAniversario(): void {
    this.idade++;
  }

  getIdade(): number {
    return this.idade;
  }
}

const pessoa = new Pessoa("Tiago", 25);

pessoa.apresentar();
pessoa.fazerAniversario();
```

```
console.log(pessoa.getIdade()); // 26
```

Aqui, Pessoa é a classe. pessoa é um objeto criado com new Pessoa(...). O nome é público porque foi declarado com public. A idade é privada, então o código externo não pode fazer pessoa.idade diretamente. O método getIdade() permite consultar esse estado de forma controlada.

3. Exercícios

Resolva na ordem. Tente escrever tudo sozinho antes de consultar o gabarito.

Exercício 1 — Classe básica

Crie uma classe Carro com uma propriedade marca. Crie também um método acelerar() que mostre uma mensagem no console. Depois crie um objeto da classe e chame o método.

Dica: Comece apenas com class, constructor, uma propriedade e um método. Não precisa usar private ainda.

Exercício 2 — Mais propriedades

Crie uma classe Aluno com nome e nota. Crie um método verificarAprovacao() que mostre "Aprovado" se a nota for maior ou igual a 7 e "Reprovado" caso contrário.

Dica: O método pode acessar diretamente as propriedades usando this.nome e this.nota.

Exercício 3 — Encapsulamento

Crie uma classe ContaBancaria com titular e saldo. O saldo deve ser privado. Crie métodos depositar(valor) e sacar(valor). Não permita sacar mais dinheiro do que existe no saldo.

Dica: Use private saldo. Dentro da classe você pode alterar o saldo normalmente.

Exercício 4 — Getters e setters

Crie uma classe Produto com nome e preco privado. Crie um getter para consultar o preço e um setter que impeça o preço de receber valores menores ou iguais a zero.

Dica: O setter pode fazer uma validação antes de executar this._preco = valor.

Exercício 5 — Métodos trabalhando juntos

Crie uma classe Carrinho que mantenha uma lista privada de produtos. Crie métodos para adicionarProduto(), removerProduto() e calcularTotal().

Dica: Você pode criar primeiro uma classe simples Produto e depois usar Produto[] dentro do Carrinho.

Exercício 6 — Herança

Crie uma classe Animal com a propriedade nome e o método emitirSom(). Depois crie Cachorro e Gato herdando de Animal.

Dica: Use extends Animal. O constructor das classes filhas pode chamar super(nome).

Exercício 7 — Polimorfismo

Aproveite o exercício anterior. Faça Cachorro e Gato implementarem emitirSom() de maneiras diferentes. Depois crie um array de Animal contendo um cachorro e um gato e percorra o array chamando emitirSom().

Dica: O ponto do exercício é perceber que você chama o mesmo método em objetos diferentes e cada classe executa sua própria implementação.

Exercício 8 — Classe abstrata

Crie uma classe abstrata Funcionario com nome e salário e um método abstrato calcularBonus(). Crie Gerente e Desenvolvedor, cada um com uma regra diferente de bônus.

Dica: Use abstract class e abstract calcularBonus(). Uma classe abstrata não deve ser instanciada diretamente.

Exercício 9 — Sistema pequeno

Crie um sistema de biblioteca. Tenha as classes Livro, Usuario e Biblioteca. A biblioteca deve permitir cadastrar livros, cadastrar usuários, emprestar um livro e devolver um livro. Um livro emprestado não pode ser emprestado novamente.

Dica: Pense nas responsabilidades: Livro sabe se está disponível; Biblioteca coordena empréstimos; Usuario representa quem pega o livro.

Exercício 10 — Desafio final

Crie um pequeno sistema de pedidos. Tenha as classes Produto, ItemPedido e Pedido. O pedido deve permitir adicionar itens, calcular subtotal, aplicar um desconto percentual e calcular o total final. O total não pode ficar negativo. Use encapsulamento e organize as responsabilidades entre as classes.

Dica: Evite colocar toda a lógica em Pedido. ItemPedido deve saber calcular seu próprio subtotal. O Pedido pode somar os subtotais e aplicar o desconto.

4. Gabarito

Compare depois de tentar. As soluções abaixo são uma forma correta de resolver, não necessariamente a única.

Exercício 1

```
class Carro {
  constructor(public marca: string) {}

  acelerar(): void {
    console.log(`${this.marca} está acelerando!`);
  }
}

const carro = new Carro("Toyota");
carro.acelerar();
```

Explicação: Carro é o molde. O objeto carro é criado com new e recebe a marca pelo construtor. O método usa o estado do objeto através de this.

Exercício 2

```
class Aluno {
  constructor(
    public nome: string,
    public nota: number
  ) {}

  verificarAprovacao(): void {
    if (this.nota >= 7) {
      console.log(`${this.nome}: Aprovado`);
    } else {
      console.log(`${this.nome}: Reprovado`);
    }
  }
}

const aluno = new Aluno("Ana", 8);
aluno.verificarAprovacao();
```

Explicação: O método usa a propriedade nota para decidir o comportamento. A classe reúne os dados do aluno e a regra de aprovação.

Exercício 3

```
class ContaBancaria {
  constructor(
    public titular: string,
    private saldo: number = 0
  ) {}

  depositar(valor: number): void {
    if (valor > 0) {
      this.saldo += valor;
    }
  }

  sacar(valor: number): void {
    if (valor > 0 && valor <= this.saldo) {
      this.saldo -= valor;
    }
  }

  getSaldo(): number {
```

```

        return this.saldo;
    }
}

const conta = new ContaBancaria("Tiago", 100);
conta.depositar(50);
conta.sacar(30);
console.log(conta.getSaldo()); // 120

```

Explicação: O saldo é privado. Isso é encapsulamento: o código externo não altera o saldo diretamente; ele precisa passar pelas operações da classe.

Exercício 4

```

class Produto {
    private _preco: number;

    constructor(
        public nome: string,
        preco: number
    ) {
        this._preco = preco;
    }

    get preco(): number {
        return this._preco;
    }

    set preco(valor: number) {
        if (valor <= 0) {
            throw new Error("Preço deve ser maior que zero.");
        }

        this._preco = valor;
    }
}

const produto = new Produto("Teclado", 150);
produto.preco = 180;
console.log(produto.preco);

```

Explicação: O getter permite leitura com `produto.preco`. O setter permite atribuição com `produto.preco = valor`, mas intercepta a alteração para validar o dado.

Exercício 5

```

class Produto {
    constructor(
        public nome: string,
        public preco: number
    ) {}
}

class Carrinho {
    private produtos: Produto[] = [];

    adicionarProduto(produto: Produto): void {
        this.produtos.push(produto);
    }

    removerProduto(nome: string): void {
        this.produtos = this.produtos.filter(
            produto => produto.nome !== nome
        );
    }

    calcularTotal(): number {

```

```

        return this.produtos.reduce(
            (total, produto) => total + produto.preco,
            0
        );
    }
}

const carrinho = new Carrinho();
carrinho.adicionarProduto(new Produto("Mouse", 80));
carrinho.adicionarProduto(new Produto("Teclado", 150));

console.log(carrinho.calcularTotal());

```

Explicação: Carrinho é responsável por administrar a coleção. Produto representa um item. Separar essas responsabilidades deixa o código mais organizado.

Exercício 6

```

class Animal {
    constructor(public nome: string) {}

    emitirSom(): void {
        console.log("Som genérico");
    }
}

class Cachorro extends Animal {
    emitirSom(): void {
        console.log("Au au!");
    }
}

class Gato extends Animal {
    emitirSom(): void {
        console.log("Miau!");
    }
}

const cachorro = new Cachorro("Rex");
const gato = new Gato("Mimi");

```

Explicação: extends cria a relação de herança. Cachorro e Gato recebem a propriedade nome de Animal e podem fornecer seu próprio comportamento.

Exercício 7

```

const animais: Animal[] = [
    new Cachorro("Rex"),
    new Gato("Mimi")
];

for (const animal of animais) {
    animal.emitirSom();
}

```

Explicação: O array é do tipo Animal, mas contém instâncias diferentes. Quando emitirSom é chamado, a implementação da classe concreta é executada. Isso é polimorfismo.

Exercício 8

```

abstract class Funcionario {
    constructor(
        public nome: string,
        public salario: number
    ) {}

    abstract calcularBonus(): number;
}

```

```

}

class Gerente extends Funcionario {
    calcularBonus(): number {
        return this.salario * 0.20;
    }
}

class Desenvolvedor extends Funcionario {
    calcularBonus(): number {
        return this.salario * 0.10;
    }
}

const funcionarios: Funcionario[] = [
    new Gerente("Ana", 10000),
    new Desenvolvedor("Carlos", 6000)
];

for (const funcionario of funcionarios) {
    console.log(funcionario.nome, funcionario.calcularBonus());
}

```

Explicação: Funcionario define uma regra que todas as classes concretas precisam cumprir. Cada classe decide como calcular seu próprio bônus.

Exercício 9

```

class Livro {
    constructor(
        public titulo: string,
        private disponivel: boolean = true
    ) {}

    estaDisponivel(): boolean {
        return this.disponivel;
    }

    emprestar(): void {
        if (!this.disponivel) {
            throw new Error("Livro já está emprestado.");
        }

        this.disponivel = false;
    }

    devolver(): void {
        this.disponivel = true;
    }
}

class Usuario {
    constructor(public nome: string) {}
}

class Biblioteca {
    private livros: Livro[] = [];
    private usuarios: Usuario[] = [];

    cadastrarLivro(livro: Livro): void {
        this.livros.push(livro);
    }

    cadastrarUsuario(usuario: Usuario): void {
        this.usuarios.push(usuario);
    }
}

```



```

    emprestarLivro(livro: Livro, usuario: Usuario): void {
        if (!this.livros.includes(livro)) {
            throw new Error("Livro não cadastrado.");
        }

        if (!this.usuarios.includes(usuario)) {
            throw new Error("Usuário não cadastrado.");
        }

        livro.emprestar();
    }

    devolverLivro(livro: Livro): void {
        livro.devolver();
    }
}

```

Explicação: A ideia principal é dividir responsabilidades. Livro controla sua disponibilidade; Biblioteca controla o cadastro e coordena a operação.

Exercício 10

```

class Produto {
    constructor(
        public nome: string,
        public preco: number
    ) {}
}

class ItemPedido {
    constructor(
        public produto: Produto,
        public quantidade: number
    ) {}

    calcularSubtotal(): number {
        return this.produto.preco * this.quantidade;
    }
}

class Pedido {
    private itens: ItemPedido[] = [];
    private descontoPercentual: number = 0;

    adicionarItem(item: ItemPedido): void {
        this.itens.push(item);
    }

    aplicarDesconto(percentual: number): void {
        if (percentual < 0 || percentual > 100) {
            throw new Error("Desconto inválido.");
        }

        this.descontoPercentual = percentual;
    }

    calcularSubtotal(): number {
        return this.itens.reduce(
            (total, item) => total + item.calcularSubtotal(),
            0
        );
    }

    calcularTotal(): number {
        const subtotal = this.calcularSubtotal();
        const desconto = subtotal * (this.descontoPercentual / 100);
    }
}

```

```

        return Math.max(0, subtotal - desconto);
    }
}

const produto = new Produto("Notebook", 3000);
const item = new ItemPedido(produto, 2);

const pedido = new Pedido();
pedido.adicionarItem(item);
pedido.aplicarDesconto(10);

console.log(pedido.calcularSubtotal()); // 6000
console.log(pedido.calcularTotal());    // 5400

```

Explicação: ItemPedido calcula seu próprio subtotal, enquanto Pedido coordena os itens e o desconto. Essa divisão evita concentrar toda a regra em uma única classe e é um passo importante para escrever POO de forma mais organizada.

5. Desafio extra — sem gabarito

Se os 10 exercícios ficaram tranquilos, tente criar sozinho um sistema de estoque com Produto, Estoque e Movimentacao. O estoque deve permitir entrada e saída, impedir quantidade negativa e mostrar o estoque atual. Tente aplicar encapsulamento e separar bem as responsabilidades.

Regra para o estudo de hoje: não copie o gabarito. Se travar, volte para a dica e tente escrever a solução de outra forma. O objetivo é fazer seu cérebro construir as classes, não reconhecer uma solução pronta.